



Universidad Nacional Autónoma De México
Facultad de ingeniería



Estructura y programación de computadoras

Trabajo final

Ing. Adara Mercado Martinez

Integrantes:

Alonso Briseño Jazmin

Ortiz Soledad Issis Fernanda

Isaias Ivan Carreon Juárez

Manzo Ramos Paola Valentina

ÍNDICE

1. Introducción
 - 1.1 Objetivo
 - General
 - Específicos
 - 1.2 Arquitectura Objetivo: Registros soportados
 - 1.3 Instrucciones mínimas: Transferencia
 - 1.4 Aritméticas
 - 1.5 Lógicas
 - 1.6 Saltos y control
 - 1.7 Directivas mínimas
 - 1.8 Modos de direccionamiento:
 - inmediato
 - registro a registro
 - Memoria directa
 - Base + desplazamiento
 - Base + índice
 - Base + índice escalado
 - Base + índice escalado + desplazamiento
 - 1.9 Soporte obligatorio byte SIB
 - opcode
 - ModRM
 - SIB
 - Displacement
 - Immediate
 - 1.10 Escalas soportadas
 - 1.11 Ejemplo
2. Ensamblador de una pasada
 - 2.1 Responsabilidades
 - Tabla de símbolos
 - Manejo de etiquetas
 - Generación del código parcial
 - Manejo de errores
3. Ensamblador de dos pasadas
 - 3.1 Primera pasada
 - Construcción de tabla de símbolos
 - Cálculo de direcciones

- Detección de redefiniciones
- 3.2 Segunda pasada
- Generación definitiva del código máquina
 - Resolución de referencias
 - Generación del archivo objeto
4. Formato objeto
- Encabezado
 - Tabla de secciones
 - Tabla de símbolos
 - Tabla de relokaciones
- 4.2 Relocaciones: ¿Qué son?, ¿Para qué sirven?, Cómo las utilizará el linker
- 4.3 Secciones mínimas
5. Mini linker
- Leer múltiples archivos objeto.
 - Resolver símbolos externos.
 - Reubicar direcciones.
 - Fusionar secciones.
 - Generar binario final.
6. Organización del equipo:
- Integrante 1: Lexer
 - Integrante 2: Parser
 - Integrante 3: Backend
 - Integrante 4: Encoder IA-32
 - Integrante 5: Formato objeto y Linker
7. Repositorio base
8. Casos de prueba
- MOV inmediato
 - Saltos
 - Referencias adelantadas
 - EXTERN
 - CALL
 - Relocaciones
 - Múltiples módulos
 - SIB
9. Bitacora IA:
- Prompts utilizados.
 - Herramientas IA utilizadas.
 - Código generado automáticamente.
 - Modificaciones manuales.
 - Errores encontrados.
 -
10. Entregables:
- Código fuente.
 - Repositorio Git.

- Reporte técnico.
- Bitácora IA.
- Casos de prueba.
- Makefile funcional.

11. Integración

- Cómo integramos los módulos.
- Cómo verificamos que funcionaban juntos.
- Problemas encontrados durante la integración.
- Resultados obtenidos.

12. Conclusión

1. Introducción

General.

Implementar un ensamblador modular para un subconjunto de IA-32 junto con un enlazador simplificado.

Específicos.

El objetivo de nuestro proyecto es desarrollar un sistema completo de traducción y construcción de programas para un subconjunto de la arquitectura IA-32. Al mismo tiempo, buscamos comprender completamente cómo funcionan e implementar:

- Lexer manual
- Parser manual
- Tablas de símbolos
- Código máquina IA-32
- Implementar ensamblado de 1 y 2 pasadas
- referencias adelantadas.
- relocalaciones
- mini linker.

¿Qué es un ensamblador?

Un ensamblador es un programa que traduce instrucciones escritas en lenguaje ensamblador a código máquina, permitiendo que el procesador pueda ejecutarlas. También administra etiquetas, símbolos y direcciones de memoria.

¿Qué es un linker?

Un linker o enlazador es una herramienta que une uno o varios archivos objeto para generar un programa ejecutable. Su función principal es resolver símbolos externos y ajustar direcciones de memoria.

¿Qué es IA-32?

IA-32 es una arquitectura de 32 bits desarrollada por Intel que define los registros, instrucciones y modos de direccionamiento utilizados por el procesador para ejecutar programas.

Importancia de comprender el proceso de traducción de código

Comprender cómo un programa se transforma en código ejecutable permite entender el funcionamiento interno de las computadoras, así como el papel que desempeñan los ensambladores, los archivos objeto y los enlazadores en la ejecución de software.

Alcance del proyecto

Este proyecto consiste en desarrollar un ensamblador para un subconjunto de la arquitectura IA-32, capaz de generar código máquina y archivos objeto, además de implementar un mini linker para enlazar módulos, resolver símbolos externos y producir un binario final.

1.2 Arquitectura Objetivo: Registros Soportados

La arquitectura IA-32 utiliza registros de propósito general de 32 bits para almacenar datos, direcciones y resultados temporales durante la ejecución de un programa. En este proyecto se implementó soporte para los siguientes registros:

Registro	Descripción
EAX	Registro acumulador. Se utiliza comúnmente en operaciones aritméticas y almacenamiento de resultados.
EBX	Registro base. Frecuentemente utilizado para direccionamiento de memoria.
ECX	Registro contador. Utilizado en ciclos, repeticiones y operaciones de desplazamiento.
EDX	Registro de datos. Empleado en operaciones aritméticas avanzadas y manejo de datos adicionales.
ESI	Registro índice fuente. Utilizado en operaciones con arreglos y cadenas de datos.
EDI	Registro índice destino. Empleado en transferencias y manipulación de bloques de memoria.
EBP	Base Pointer. Se utiliza para acceder a variables y parámetros dentro de la pila.
ESP	Stack Pointer. Apunta al tope de la pila y se actualiza durante operaciones PUSH y POP.

Instrucciones mínimas soportadas

El ensamblador implementado reconoce un subconjunto de instrucciones de la arquitectura IA-32. Estas instrucciones se agrupan en instrucciones de transferencia, aritméticas, lógicas y de control de flujo.

Transferencia de datos

Instrucción	Descripción	Número de operandos	Ejemplos
MOV	Copia un dato desde un origen hasta un destino	2	MOV EAX,EBX
PUSH	Inserta un valor en la pila	1	PUSH EAX
POP	Extrae un valor en la pila	1	POP EBX
LEA	Carga una dirección efectiva en un registro	2	LEA EAX [EBX+4]

Instrucciones aritméticas

Instrucción	Descripción	Número de operandos	Ejemplo
ADD	Suma dos operandos	2	ADD EAX, EBX
SUB	Resta el segundo operando al primero	2	SUB EAX, EBX
INC	Incrementa un operando en una unidad	1	INC EAX
DEC	Decrementa un operando en una unidad	1	DEC EAX
CMP	Compara dos operandos y modifica las banderas	2	CMP EAX, EBX
NEG	Cambia el signo del	1	NEG EAX

	operando mediante complemento a dos		
MUL	Multiplica el acumulador por el operando indicado	1	MUL EBX
DIV	Divide el acumulador extendido entre el operando indicado	1	DIV EBX

Instrucciones lógicas

Instrucciones	Descripción	Número de operandos	Ejemplos
AND	Realiza una operación lógica AND bit a bit	2	AND EAX, EBX
OR	Realiza una operación lógica OR bit a bit	2	OR EAX, EBX
XOR	Realiza una operación lógica XOR bit a bit	2	XOR EAX, EBX
NOT	Invierte los bits del operando	1	NOT EAX

Salto y control

Instrucción	Descripción	Número de operandos	Ejemplos
JMP	Realiza un salto incondicional	1	JMP etiqueta
JE	Salta si la comparación anterior resultó igual	1	JE igual
JNE	Salta si la comparación anterior resultó diferente	1	JNE distinto

JG	Salta si el resultado fue mayor	1	JG mayor
JL	Salta si el resultado fue menor	1	JL menor
JGE	Salta si el resultado fue mayor o igual	1	JGE mayor igual
JLE	Salta si el resultado fue menor o igual	1	JLE menor igual
CALL	Llama a una subrutina	1	CALL funcion
RET	Regresa de una subrutina	0	RET
NOP	No realiza operación	0	NOP
INT	Genera una interrupción de software	1	INT 80h

Implementación del reconocimiento de instrucciones

El reconocimiento de instrucciones se realiza en el parser mediante una tabla de mnemónicos que asocia cada instrucción del lenguaje ensamblador con un código interno de operación. De esta forma, instrucciones como **MOV**, **ADD**, **JMP** o **CALL** pueden ser identificadas y procesadas por el sistema.

Una vez reconocida la instrucción, el parser verifica que la cantidad de operandos sea correcta y analiza el tipo de cada operando, ya sea un registro, un valor inmediato, una dirección de memoria o una etiqueta.

Finalmente, la información obtenida se almacena en una estructura interna denominada AST (Árbol de Sintaxis Abstracta), la cual será utilizada posteriormente por el ensamblador para generar el código máquina correspondiente.

1.7 Directivas mínimas

Las directivas son instrucciones especiales que no generan código máquina directamente, sino que permiten organizar el programa, definir datos y administrar símbolos dentro del ensamblador.

SECTION

Permite seleccionar la sección de trabajo actual del programa.

Ejemplo:

```
SECTION .text
```

GLOBAL

Declara un símbolo como global para que pueda ser utilizado por otros módulos.

Ejemplo:

```
GLOBAL main
```

EXTERN

Declara un símbolo definido en otro módulo.

Ejemplo:

```
EXTERN printf
```

DB (Define Byte)

Define uno o más datos de 8 bits.

Ejemplo:

```
mensaje DB 72,101,108,108,111
```

DW (Define Word)

Define uno o más datos de 16 bits.

Ejemplo:

```
valor DW 1000
```

DD (Define Double Word)

Define uno o más datos de 32 bits.

Ejemplo:

numero DD 123456

ORG

Establece la dirección base a partir de la cual se ensamblará el programa.

Ejemplo:

ORG 1000h

EQU

Permite definir constantes simbólicas.

Ejemplo:

MAXIMO EQU 100

RESB (Reserve Byte)

Reserva una cantidad determinada de bytes.

Ejemplo:

buffer RESB 64

RESW (Reserve Word)

Reserva una cantidad determinada de palabras de 16 bits.

Ejemplo:

datos RESW 32

RESB (Reserve Double Word)

Reserva una cantidad determinada de dobles palabras de 32 bits.

Ejemplo:

arreglo RESD 16

1.8 Modos de direccionamiento

Los modos de direccionamiento determinan la forma en que una instrucción accede a sus operandos. El ensamblador implementado soporta los principales modos de direccionamiento de la arquitectura IA-32.

Direccionamiento inmediato

El operando se encuentra directamente dentro de la instrucción.

Ejemplo:

```
MOV EAX, 10
```

Direccionamiento registro a registro

La transferencia de datos se realiza entre registros.

Ejemplo:

```
MOV EAX, EBX
```

Memoria directa

La dirección de memoria se especifica explícitamente.

Ejemplo:

```
MOV EAX, [1000]
```

Base + desplazamiento

Se utiliza un registro base y un desplazamiento constante.

Ejemplo:

```
MOV EAX, [EBP+4]
```

Base + índice

La dirección efectiva se obtiene sumando un registro base y un registro índice.

Ejemplo:

```
MOV EAX, [EBX+ECX]
```

Base + índice escalado

El registro índice es multiplicado por una escala antes de sumarse al registro base.

Ejemplo:

```
MOV EAX, [EBX+ECX*4]
```

Base + índice escalado + desplazamiento

Además del registro base y el índice escalado, se agrega un desplazamiento constante.

Ejemplo:

```
MOV EAX, [EBX+ECX*4+8]
```

Estos modos de direccionamiento permiten acceder eficientemente a variables, arreglos, estructuras de datos y posiciones específicas de memoria dentro de un programa.

1.9 Soporte obligatorio SIB

El ensamblador implementa soporte para el byte SIB (Scale Index Base), utilizado en la arquitectura IA-32 para calcular direcciones de memoria más complejas. Este mecanismo permite combinar un registro base, un registro índice, un factor de escala y un desplazamiento para obtener una dirección efectiva.

El soporte SIB es especialmente útil para acceder a arreglos, estructuras y bloques de memoria de manera eficiente.

La estructura general de este direccionamiento es:

$[Base + Índice * Escala + Desplazamiento]$

Ejemplo:

```
MOV EAX, [EBX+ECX*4+8]
```

En este ejemplo:

- EBX actúa como registro base.
- ECX actúa como registro índice.
- 4 corresponde al factor de escala.
- 8 corresponde al desplazamiento.

Este tipo de direccionamiento es reconocido y validado por el parser durante el análisis de operandos de memoria.

1.10 Escalas soportadas

Como parte del soporte SIB, el ensamblador permite utilizar los factores de escala definidos por la arquitectura IA-32.

Las escalas soportadas son:

Escala	Significado
1	El índice se utiliza sin modificación
2	El índice se multiplica por 2
4	El índice se multiplica por 4
8	El índice se multiplica por 8

Cualquier otro valor es considerado inválido y es reportado como un error durante el análisis sintáctico. Estas escalas permiten acceder eficientemente a elementos de arreglos cuyos tamaños son múltiplos de 1, 2, 4 u 8 bytes.

1.11 Ejemplo

A continuación se muestra un ejemplo del direccionamiento SIB implementado en el proyecto:

```
MOV EAX, [EBX+ECX*4+8]
```

En esta instrucción, la dirección efectiva se calcula utilizando los siguientes componentes:

Componente	Valor
Registro base	EBX
Registro índice	ECX
Escala	4
Desplazamiento	8

La dirección resultante se obtiene mediante la siguiente operación:

$\text{Dirección} = \text{EBX} + (\text{ECX} \times 4) + 8$

Una vez calculada la dirección efectiva, el contenido almacenado en dicha posición de memoria es transferido al registro EAX.

Este ejemplo demuestra el uso completo del direccionamiento Base + Índice Escalado + Desplazamiento, uno de los modos de direccionamiento más utilizados en la arquitectura IA-32.

2. Ensamblador de una pasada

El ensamblador de una pasada realiza el proceso de traducción del programa fuente recorriendo el árbol de sintaxis abstracta (AST) una sola vez. Durante este recorrido se procesan instrucciones, directivas, etiquetas y símbolos, generando simultáneamente la información necesaria para el ensamblado.

La implementación se encuentra en la función `assemble_one_pass()`, la cual inicializa las estructuras internas del ensamblador, procesa cada nodo del AST y finalmente resuelve las referencias pendientes mediante el mecanismo de fixups.

2.1 Responsabilidades

Tabla de símbolos

La tabla de símbolos almacena la información de las etiquetas y símbolos utilizados dentro del programa. Cada entrada contiene el nombre del símbolo, su dirección, la sección a la que pertenece y su tipo.

Los tipos de símbolos soportados son:

- Locales (LOCAL)
- Globales (GLOBAL)
- Externos (EXTERN)
- Constantes definidas con EQU

La tabla de símbolos permite localizar direcciones y resolver referencias durante el proceso de ensamblado.

Manejo de etiquetas

Cuando el ensamblador encuentra una etiqueta, calcula su dirección actual utilizando la sección activa y el desplazamiento correspondiente. Posteriormente, la etiqueta es registrada dentro de la tabla de símbolos para que pueda ser utilizada por instrucciones de salto, llamadas a funciones o referencias a datos.

Generación de código parcial

Durante la única pasada del ensamblador se procesan las instrucciones y directivas del programa. Conforme se recorren los nodos del AST, los bytes generados son almacenados en la sección correspondiente.

Para ello se utilizan funciones encargadas de escribir bytes individuales, arreglos de bytes y valores de 32 bits dentro de las secciones del programa.

Manejo de errores

El ensamblador verifica diversas condiciones durante el procesamiento del programa, entre ellas:

- Redefinición de símbolos.
- Secciones desconocidas.
- Operandos inválidos.
- Desbordamiento de secciones.
- Referencias a símbolos inexistentes.

Si se detecta algún error, este es reportado al usuario indicando la causa correspondiente.

3. Ensamblador de dos pasadas

El ensamblador de dos pasadas divide el proceso de ensamblado en dos etapas independientes. Este enfoque permite resolver correctamente las referencias adelantadas, calcular direcciones de memoria con precisión y generar el código máquina definitivo.

La implementación se realiza mediante las funciones `first_pass()` y `second_pass()`, las cuales recorren el árbol de sintaxis abstracta (AST) en momentos diferentes del proceso de ensamblado.

3.1 Primera pasada

Durante la primera pasada no se genera código máquina. Su objetivo principal es recopilar información necesaria para el ensamblado final.

Construcción de la tabla de símbolos

En esta etapa se identifican todas las etiquetas y símbolos definidos en el programa. Cada símbolo es registrado junto con su dirección, sección y tipo correspondiente.

Esta información es almacenada en la tabla de símbolos para ser utilizada posteriormente durante la generación de código.

Cálculo de direcciones

A medida que se recorren las instrucciones y directivas, el ensamblador calcula el espacio que ocupará cada elemento dentro de la memoria.

Para ello se consideran:

- Instrucciones.
- Datos definidos mediante DB, DW y DD.
- Espacios reservados mediante RESB, RESW y RESD.
- Direcciones base establecidas mediante ORG.

De esta forma se obtiene la dirección exacta de cada símbolo antes de generar el código definitivo.

Detección de redefiniciones

Durante la construcción de la tabla de símbolos se verifica que un mismo símbolo no sea definido más de una vez. Si se detecta una redefinición, el ensamblador genera un mensaje de error y detiene el proceso de ensamblado.

3.2 Segunda pasada

Una vez conocida la ubicación de todos los símbolos, se realiza una segunda lectura del programa para generar el código final.

Generación definitiva de código máquina

En esta etapa se procesan nuevamente las instrucciones y directivas del AST. Los bytes generados son almacenados en las secciones correspondientes del programa.

Las secciones soportadas por el ensamblador son:

- .text
- .data
- .bss

Resolución de referencias

Las referencias a etiquetas y símbolos son resueltas utilizando la información almacenada en la tabla de símbolos durante la primera pasada.

Esto permite sustituir referencias simbólicas por las direcciones reales correspondientes.

Generación de fixups

Cuando una referencia no puede resolverse inmediatamente, el ensamblador crea un registro de fixup que almacena la información necesaria para completar la dirección posteriormente.

Al finalizar la segunda pasada, todos los fixups son procesados y las referencias pendientes son actualizadas con los valores correctos.

Este mecanismo permite soportar referencias adelantadas y símbolos definidos en posiciones posteriores del programa.

4. Formato objeto

El proyecto implementa un formato objeto propio identificado mediante la firma `IA32OBJ`. Este formato permite almacenar la información generada por el ensamblador, incluyendo secciones, símbolos y relocalaciones. Su propósito es conservar la información necesaria para que posteriormente pueda ser leída por el linker y utilizada en la construcción del binario final.

El formato objeto se implementa mediante los archivos `object.h` y `object.c`, donde se definen las estructuras principales del archivo objeto y las funciones encargadas de escribirlo y leerlo.

4.1 Encabezado

El encabezado del archivo objeto almacena la información general del archivo. Está representado por la estructura `ObjHeader`.

El encabezado contiene:

- Firma de identificación del archivo.
- Número de secciones.
- Número de símbolos.
- Número de relocalaciones.

La firma utilizada es:

`IA32OBJ`

Esta firma permite verificar que el archivo leído corresponde al formato objeto definido por el proyecto.

4.2 Tabla de secciones

La tabla de secciones almacena la información de cada sección del programa. Cada entrada se representa mediante la estructura `ObjSectionHeader`.

Cada sección contiene:

- Nombre de la sección.
- Tipo de sección.
- Tamaño.
- Offset dentro del archivo objeto.
- Dirección base.

Las secciones soportadas son:

Sección	Descripción
.text	Contiene el código generado por el ensamblador.
.data	Contiene datos inicializados.
.bss	Contiene datos reservados sin inicializar.

En el archivo objeto se escriben físicamente los datos correspondientes a `.text` y `.data`. La sección `.bss` sólo conserva su tamaño y metadatos, ya que representa memoria reservada sin inicializar.

4.3 Tabla de símbolos

La tabla de símbolos almacena la información de las etiquetas y símbolos detectados durante el ensamblado. Cada símbolo se representa con la estructura `ObjSymbol`.

Cada entrada contiene:

- Nombre del símbolo.
- Valor o dirección.
- Sección asociada.
- Tipo de símbolo.
- Estado de definición.

Los símbolos pueden ser locales, globales, externos o constantes definidas mediante `EQU`.

Esta tabla es fundamental para que el linker pueda resolver referencias entre distintos archivos objeto.

4.4 Tabla de relocalaciones

La tabla de relocalaciones almacena las referencias que necesitan ser ajustadas posteriormente. Cada relocalación se representa mediante la estructura `ObjRelocation`.

Cada entrada contiene:

- Nombre del símbolo referenciado.
- Sección donde aparece la referencia.
- Offset dentro de la sección.
- Tipo de relocalación.
- Valor adicional o addend.

Estas relocalaciones se generan a partir de los fixups creados durante el ensamblado.

4.5 Relocaciones

Una relocalación indica que una dirección o referencia simbólica debe ser corregida cuando se conozca la ubicación final del símbolo. Este proceso es necesario porque, durante el ensamblado, algunas direcciones todavía no pueden determinarse de forma definitiva.

En este proyecto, las relocalaciones permiten que referencias a símbolos externos o adelantados puedan resolverse posteriormente por el linker.

4.6 Lectura y escritura del archivo objeto

La función `write_object_file()` se encarga de generar el archivo objeto. Para ello escribe, en orden:

- Encabezado.
- Tabla de secciones.
- Tabla de símbolos.
- Tabla de relocalaciones.
- Datos de la sección `.text`.
- Datos de la sección `.data`.

Por otro lado, la función `read_object_file()` permite leer un archivo objeto previamente generado. Esta función valida la firma `IA32OBJ`, reconstruye las secciones, carga la tabla de símbolos, recupera las relocalaciones y lee los datos de las secciones correspondientes.

Con esto, el proyecto cuenta con un formato objeto propio que permite almacenar la salida del ensamblador de forma estructurada y preparada para el proceso de enlazado.

5. Mini linker

El proyecto implementa un mini linker para combinar uno o más archivos objeto generados por el ensamblador. Su función principal es leer los archivos objeto, resolver símbolos externos, aplicar relokaciones, fusionar secciones y generar un binario final.

El linker se encuentra implementado en el archivo `linker.c` y trabaja junto con el formato objeto definido en `object.h` y `object.c`.

5.1 Lectura de múltiples archivos objeto

El linker permite recibir varios archivos objeto como entrada. Cada archivo es leído mediante la función `read_object_file()`, la cual reconstruye sus secciones, símbolos y relokaciones.

El programa permite especificar un archivo de salida mediante la opción `-o` y una dirección base mediante la opción `-b`.

Ejemplo de uso:

```
./linker -o program.bin -b 0x00000000 main.obj lib.obj
```

5.2 Resolución de símbolos externos

Durante el enlace, el linker construye una tabla global de símbolos. En esta tabla se registran los símbolos globales definidos por cada archivo objeto y los símbolos externos que deben resolverse.

Si un símbolo externo no tiene una definición correspondiente en otro módulo, el linker reporta un error y detiene el proceso.

También se detectan errores cuando un mismo símbolo global aparece definido más de una vez.

5.3 Reubicación de direcciones

Después de calcular las direcciones finales de las secciones, el linker aplica las relokaciones pendientes.

El proyecto soporta dos tipos principales de fixups:

- `FIX_ABS32`: relokación absoluta de 32 bits.
- `FIX_REL32`: relokación relativa de 32 bits.

En las relokaciones absolutas, se escribe directamente la dirección final del símbolo. En las relokaciones relativas, se calcula la distancia entre la dirección del símbolo y el final de la instrucción, lo cual es necesario para saltos y llamadas relativas.

5.4 Fusión de secciones

El linker fusiona las secciones equivalentes de todos los archivos objeto.

Las secciones fusionadas son:

- `.text`
- `.data`
- `.bss`

Primero se calcula el tamaño total de cada sección y después se asignan offsets individuales para cada archivo objeto. Posteriormente, el contenido de las secciones `.text` y `.data` se copia en buffers unificados.

La sección `.bss` no se escribe directamente en el binario final, ya que representa memoria reservada sin inicializar.

5.5 Generación del binario final

Una vez resueltos los símbolos y aplicadas las relocalaciones, el linker genera un archivo binario final. Este archivo contiene la unión de las secciones `.text` y `.data`.

Además, el linker genera un archivo de mapa con extensión `.map`, donde se muestra información útil para depuración, como:

- Dirección base del programa.
- Tamaños finales de las secciones.
- Distribución de cada archivo objeto.
- Tabla global de símbolos.
- Símbolos locales y constantes.

5.6 Resultado del enlazado

El resultado final del proceso de enlazado es un binario ejecutable simplificado, acompañado de un archivo de mapa que permite revisar cómo fueron ubicadas las secciones y los símbolos dentro del programa.

Este mini linker demuestra el funcionamiento básico de un enlazador real, ya que realiza lectura de archivos objeto, resolución de símbolos, reubicación de direcciones y generación de una imagen binaria final.

6. Organización del equipo

Lexer y Tokens

Responsable de la lectura de archivos ASM, tokenización, reconocimiento de números, registros, comentarios y validación léxica.

Parser y Representación Interna

Responsable del análisis sintáctico, validación de instrucciones, operandos, directivas y construcción del AST.

Backend del Ensamblador

Responsable de la tabla de símbolos, ensamblador de una pasada, ensamblador de dos pasadas, fixups y referencias adelantadas.

Encoder IA-32

Responsable de la generación de opcodes, ModRM, SIB, displacement e immediate para las instrucciones soportadas.

Formato Objeto y Linker

Responsable de la implementación del formato objeto, relocalaciones, resolución de símbolos y generación del binario final.

El proyecto se desarrolló utilizando una estructura modular organizada en directorios para separar encabezados, código fuente, ejemplos y pruebas.

7. Repositorio base.

Estructura principal:

ia32-assembler/

├── include/

├── src/

├── examples/

├── test/

├── Makefile

└── README.md

La separación modular permitió desarrollar y probar cada componente de forma independiente antes de realizar la integración completa del sistema.

8. Casos de prueba

```

PS G:\Projects\proyecto-final-picafresas\ia32-assembler> python test/run_t
[+] Iniciando pruebas positivas del ensamblador...
[OK] [Assembler] test_instructions (2-Pass)      -> PASSED
[OK] [Assembler] test_instructions (1-Pass)      -> PASSED
[OK] [Assembler] test_addressing (2-Pass)        -> PASSED
[OK] [Assembler] test_addressing (1-Pass)        -> PASSED
[OK] [Assembler] test_one_pass (Forward Ref)     -> PASSED
[OK] [Assembler] test_directives                 -> PASSED

[+] Iniciando pruebas negativas del ensamblador (deben fallar)...
[OK] [Assembler] test_error_syntax (Syntax err)  -> PASSED
[OK] [Assembler] test_error_semantic (Semantic err) -> PASSED

[+] Iniciando pruebas del Linker...
[OK] [Linker] Link Multi-Module                  -> PASSED
[OK] [Linker] Link Unresolved Symbol             -> PASSED

[+] Iniciando pruebas de Objdump...
[OK] [Objdump] Objdump test_directives           -> PASSED

=====
                        RESUMEN DE LA SUITE DE PRUEBAS
=====
Total de Pruebas: 11
Aprobadas:        11
Fallidas:         0

=====
Prueba                                | Categoría | Resultado
-----
test_instructions (2-Pass)             | Assembler | PASSED
test_instructions (1-Pass)             | Assembler | PASSED
test_addressing (2-Pass)               | Assembler | PASSED

```



```
[object] Archivo Objeto - lib.obj - escrito exitosamente
PS G:\Projects\proyecto-final-picafresas\ia32-assembler> .\objdump.exe main.obj
=== Herramienta Objdump IA-32 ===
Integrante 5: Formato Objeto y Linker
```

```
=====
ARCHIVO OBJETO: main.obj
=====
```

```
CABECERA:
  Magic:      IA32OBJ
  Secciones:  3
  Simbolos:   4
  Relocs:     2
```

```
SECCIONES:
  Idx  Nombre      Tamano      Dir Base
  ----
  0    .text       22 bytes    0x00000000
  1    .data       4 bytes     0x00000000
  2    .bss        0 bytes     0x00000000
```

```
TABLA DE SIMBOLOS (4 entradas):
  Nombre      Valor      seccion  Tipo      Definido
  -----
  _start      00000000    .text    GLOBAL    si
  print_msg   00000000    .text    EXTERN    no
  calculo     00000000    .text    EXTERN    no
  resultado   00000000    .data    LOCAL     si
```

```
TABLA DE RELOCACIONES (2 entradas):
  Simbolo      seccion  Offset  Tipo      Fin Instr / Addend
```

```
• PS G:\Projects\proyecto-final-picafresas\ia32-assembler> make clean
if exist assembler del assembler
if exist assembler.exe del assembler.exe
if exist linker del linker
if exist linker.exe del linker.exe
if exist objdump del objdump
if exist objdump.exe del objdump.exe
• PS G:\Projects\proyecto-final-picafresas\ia32-assembler> make
gcc -Wall -Wextra -Iinclude -std=c11 src/lexer.c src/parser.c src/assembler.c src/object.c src/main.c -o assembler
gcc -Wall -Wextra -Iinclude -std=c11 src/lexer.c src/parser.c src/assembler.c src/object.c src/linker.c -o linker
gcc -Wall -Wextra -Iinclude -std=c11 src/lexer.c src/parser.c src/assembler.c src/object.c src/linker.c src/objdump.c -o objdump
• PS G:\Projects\proyecto-final-picafresas\ia32-assembler> .\assembler.exe examples/main.asm -o main.obj
=== Ensamblador IA-32 ===
Integrante 1: Lexer
Integrante 2: Parser
Integrante 3: Backend
Integrante 4: Encoder IA-32
Integrante 5: Formato Objeto y Linker
[lexer] 49 tokens generados
[parser] 13 nodos, 0 error(es)
[assembler] modo: dos pasadas

[assembler] iniciando primera pasada...
[assembler] iniciando segunda pasada...
[assembler] ensamblado exitoso
[encoder] Integrante 4 validado: opcodes, ModR/M, SIB, displacement, immediate, bytes, fixups

=== TABLA DE SIMBOLOS (4 entradas) ===
  Nombre      Valor      Seccion  Tipo      Definido
  -----
```

```

[object] Archivo objeto 'main.obj' escrito exitosamente
PS G:\Projects\proyecto-final-picafresas\ia32-assembler> .\assembler.exe examples/lib.asm -o lib.obj
=== Ensamblador IA-32 ===
Integrante 1: Lexer
Integrante 2: Parser
Integrante 3: Backend
Integrante 4: Encoder IA-32
Integrante 5: Formato Objeto y Linker
[lexer] 65 tokens generados
[parser] 15 nodos, 0 error(es)
[assembler] modo: dos pasadas

[assembler] iniciando primera pasada...
[assembler] iniciando segunda pasada...
[assembler] ensamblado exitoso
[encoder] Integrante 4 validado: opcodes, ModR/M, SIB, displacement, immediate, bytes, fixups

=== TABLA DE SIMBOLOS (3 entradas) ===
Nombre          Valor      Seccion  Tipo      Definido
-----
calculo          00000000 .text    GLOBAL    si
print_msg        00000000 .text    GLOBAL    si
mensaje          00000000 .data     LOCAL     si
=====

=== SECCION .text (20 bytes) ===
0000: 03 C3 81 E8 32 00 00 00 F7 EB F7 F9 C3 B9 00 00
0010: 00 00 90 C3
=====

```

```

issisortiz@MacBook-Air-de-Issis ia32-assembler % make clean
make
./assembler examples/test.asm
rm -f assembler
gcc -Wall -Wextra -Iinclude -std=c11 src/lexer.c src/parser.c src/assembler.c src/main.c -o assembler
=== Ensamblador IA-32 ===
Integrante 1: Lexer
Integrante 2: Parser
Integrante 3: Backend

[lexer] 107 tokens generados
[parser] 22 nodos, 0 error(es)
[assembler] modo: dos pasadas

[assembler] iniciando primera pasada...
[assembler] iniciando segunda pasada...
[assembler] ensamblado exitoso

=== TABLA DE SIMBOLOS (6 entradas) ===
Nombre          Valor      Seccion  Tipo      Definido
-----
main             00000000 .text    GLOBAL    si
printf           00000000 .text    EXTERN    si
fin              00000042 .text    LOCAL     si
mensaje          00000000 .data     LOCAL     si
valor            00000005 .data     LOCAL     si
buffer           00000000 .bss      LOCAL     si
=====

=== SECCION .text (12 bytes) ===
0000: 90 90 90 90 90 90 90 90 90 90 90 90
=====

=== SECCION .data (9 bytes) ===
0000: 48 6F 6C 61 00 D2 04 00 00
=====

```

9.1 Herramientas IA utilizadas

Durante el desarrollo del proyecto se utilizó la herramienta de inteligencia artificial Gemini como apoyo para la comprensión de conceptos teóricos, revisión de código, detección de errores y elaboración de documentación técnica.

9.2 Prompts utilizados

Algunos de los prompts empleados durante el desarrollo fueron:

- “Implementa en C un lexer manual para un ensamblador IA-32.”
- “Implementa un parser recursivo descendente en C para un ensamblador IA-32.”
- “Implementa un encoder IA-32 en C.”

9.3 Código generado automáticamente

La herramienta fue utilizada para generar ejemplos, fragmentos de código de referencia y propuestas de implementación relacionadas con:

- Manejo de tablas de símbolos.
- Construcción de estructuras de datos.
- Gestión de fixups.
- Ejemplos de codificación IA-32.
- Lectura y escritura de archivos objeto.
- Documentación técnica del proyecto.

Todo el código sugerido fue revisado antes de incorporarse al proyecto.

9.4 Modificaciones realizadas manualmente

Las siguientes actividades fueron realizadas directamente por los integrantes del equipo:

- Integración entre Lexer, Parser, Backend y Encoder IA-32.
- Ajuste de las estructuras de datos (Token, ASTNode, Symbol, Fixup y Section) para que fueran compatibles entre módulos.
- Implementación y corrección de los modos de direccionamiento soportados por el proyecto (inmediato, registro, memoria directa, base+desplazamiento, base+índice y SIB).
- Adaptación de la codificación de instrucciones IA-32 mediante la generación de Opcode, ModRM y SIB.
- Implementación y validación del manejo de etiquetas, símbolos locales, símbolos globales y símbolos externos.
- Desarrollo y depuración del sistema de fixups para referencias adelantadas y relocalaciones internas.

- Implementación de los algoritmos de ensamblado de una pasada y dos pasadas.
- Corrección de errores de compilación, validación sintáctica y manejo de errores léxicos y semánticos.
- Realización de pruebas funcionales con distintos programas ASM para verificar la generación correcta de código máquina y la resolución de símbolos.
- Organización y modularización del código para mantener una arquitectura consistente entre los distintos integrantes.

9.5 Errores encontrados

Durante el desarrollo del proyecto se identificaron y corrigieron diversos problemas, entre ellos:

- Errores en el reconocimiento de tokens.
- Problemas de validación sintáctica.
- Manejo incorrecto de referencias adelantadas.
- Resolución de símbolos externos.

9.6 Contribución de la IA al proyecto

La inteligencia artificial fue utilizada únicamente como herramienta de apoyo para consulta, documentación y revisión técnica. Las decisiones de diseño, implementación, integración y validación final fueron realizadas por los integrantes del equipo.

10. Entregables

Como resultado del proyecto se entrega el siguiente conjunto de componentes:

- Código fuente completo del ensamblador IA-32.
- Código fuente del formato objeto.
- Código fuente del mini linker.
- Repositorio del proyecto.
- Reporte técnico.
- Bitácora IA.
- Casos de prueba.
- Makefile para compilación.
- Archivos objeto generados durante las pruebas.
- Binario final generado por el linker.
- Archivo de mapa de enlace (.map).

Todos los entregables fueron desarrollados siguiendo los requerimientos establecidos para el proyecto integrador.

integración

ASM



Lexer



Parser



Tabla de símbolos



Encoder IA-32



Archivo objeto



Linker



Binario final

La integración del proyecto consistió en unir todos los módulos desarrollados de manera independiente para formar un sistema completo capaz de traducir programas escritos en lenguaje ensamblador IA-32 hasta generar un binario final.

El flujo de procesamiento comienza con el lexer, encargado de leer el archivo ASM y convertirlo en una secuencia de tokens. Posteriormente, el parser analiza dichos tokens y

construye un Árbol de Sintaxis Abstracta (AST) que representa internamente las instrucciones, directivas y operandos del programa.

A continuación, el backend del ensamblador procesa el AST para construir la tabla de símbolos, administrar las secciones del programa y generar los fixups necesarios para resolver referencias adelantadas. Durante esta etapa también se realiza el ensamblado de una o dos pasadas, según el modo de operación seleccionado.

Una vez generado el código máquina, el módulo de formato objeto almacena la información del programa en un archivo objeto que contiene secciones, símbolos y relocalaciones. Finalmente, el mini linker lee uno o más archivos objeto, resuelve símbolos externos, aplica las relocalaciones necesarias, fusiona las secciones correspondientes y genera el binario final.

Para verificar la correcta integración de todos los módulos se realizaron pruebas utilizando distintos programas ensamblador que incluían instrucciones, directivas, referencias adelantadas, símbolos externos y direccionamiento SIB. Asimismo, se probaron escenarios de múltiples módulos para validar el funcionamiento del formato objeto y del linker.

Durante la integración se presentaron algunos problemas relacionados con la resolución de símbolos, el manejo de fixups y la aplicación de relocalaciones. Estos inconvenientes fueron corregidos mediante pruebas iterativas y depuración de los distintos componentes.

Los resultados obtenidos demostraron que los módulos desarrollados funcionan correctamente de manera conjunta, permitiendo ensamblar programas, generar archivos objeto y producir un binario final mediante el proceso de enlazado.

11. Conclusiones

En este proyecto se desarrolló un ensamblador modular para un subconjunto de la arquitectura IA-32, permitiendo comprender de manera práctica el proceso de traducción de código ensamblador a una representación ejecutable.

Se implementaron correctamente las etapas de análisis léxico, análisis sintáctico, tabla de símbolos, ensamblado de una y dos pasadas, manejo de referencias adelantadas, formato objeto y mini linker. Además, se incorporó soporte para distintos modos de direccionamiento, incluyendo direccionamiento SIB.

Las pruebas realizadas demostraron el correcto funcionamiento e integración de los módulos desarrollados. Gracias a este proyecto se fortalecieron conocimientos sobre arquitectura de computadoras, programación de sistemas y el funcionamiento interno de ensambladores, archivos objeto y enlazadores.